

1a. Loop Invariant: $0 \leq i \leq n \wedge t \geq i$

1b. BASE CASE: does the LI hold before the loop?

$0 \leq i \leq n \wedge t \geq i$

the LI holds, given that $i == 0$ and $t == 0$. $(i < n) = (0 < n) = (0 \geq n)$ by precondition

1c. INDUCTION: assume the LI holds at iteration k , does it hold at iteration $k+1$?

- $t_{\text{new}} = t + i$
- $i_{\text{new}} = i + 1$
- $t_{\text{new}} = t + i_{\text{new}} - 1$

- i_{new} also holds; $0 \leq i < n$ at iteration k .
- If $i == n$ at iteration k , there would be no iteration $k+1$

1d. $LI \wedge !b \Rightarrow$ postcondition

- $0 \leq i \leq n \wedge t \geq i \wedge !(i < n)$
- $\Rightarrow 0 \leq i == n \wedge t \geq i$
- $0 == i == n \wedge t \geq i$
- $n == 0 \wedge t \geq 0$
- $\Rightarrow t \geq n \wedge t == n * (n + 1) / 2$
- $\Rightarrow t == 0 * (0 + 1) / 2 == 0$
- $\Rightarrow t == n * (n + 1) / 2$ //by precondition

3e. Does the loop terminate?

- Define $D == n - i$ // initially $i == 0$ and $n \geq 0$, $D \geq 0$
- Loop can be rewritten as: $(\text{While } D \geq 0) \{ D_{\text{new}} = n - (i + 1) = (n - i) - 1 = D - 1 \}$
- D decreases by 1 with each step.
- D eventually reaches 0.
- $D == 0 \Rightarrow$ loop exit condition $i == 0$
- When $D == 0$, loop exits

2a. Test 1: input is $n==4$ and output is $\text{root} == 2$

Before loop: $\text{root} == 0$ and $a == 4$

Iteration 1: $\text{root} == 1$ and $a == 3$

Iteration 2: $\text{root} == 2$ and $a == 0$

Loops ends and root is returned

Test 2: input is $n==25$ and output is $\text{root} == 5$

Before loop: $\text{root} == 0$ and $a == 25$

Iteration 1: $\text{root} == 1$ and $a == 24$

Iteration 2: $\text{root} == 2$ and $a == 21$

Iteration 3: $\text{root} == 3$ and $a == 16$

Iteration 4: $\text{root} == 4$ and $a == 9$

Iteration 5: $\text{root} == 5$ and $a == 0$

Loops ends and root is returned

Test 3: input is $n == 49$ and output is $root == 7$

Before loop: $root == 0$ and $a == 49$

Iteration 1: $root == 1$ and $a == 48$

Iteration 2: $root == 2$ and $a == 45$

Iteration 3: $root == 3$ and $a == 40$

Iteration 4: $root == 4$ and $a == 33$

Iteration 5: $root == 5$ and $a == 24$

Iteration 6: $root == 6$ and $a == 13$

Iteration 7: $root == 7$ and $a == 0$

Loops ends and root is returned

2b. Test 4: input is $n == 8$ and output is 3

(Step by step analysis is not necessary here as we know the square root of 8 is not 3, or even a whole number).

The bug here is that numbers which do not have perfect squares provide the incorrect output.

The part of the specification that is violated is “ensures $root * root == n$ ”, as in a scenario such as Test 4, the code cannot verify due to the inability to ensure that $3*3==8$.

2c. To fix the bug, I added:

```
    If ( $a < 0$ ) {  
         $a := 0$ ;  
         $root := root - 1$ ;  
    }
```

Doing test 4 again, if we give the input 8, the output is now 2. This passes the test as 2 is the first whole number under 3, which is the first whole number that goes over 8 when squared.

2d. To update the specification, I changed the post condition from $root * root \leq n \parallel n < 0$. This reflects the situation where the root times itself is less than equal to n , less than being important to give correct output, and the “or $n < 0$ ” to ensure edge cases where the root squared may be greater than n , but n is itself still greater than n .

2e. The dafny code does not verify. However, it is likely due to the lack of invariants and decrementing function rather than an actual bug.

2f. To make my dafny code verify, I had to following invariants and decrementing function:

Invariant $0 \leq a \leq n \parallel 0 > a \geq n$

Invariant $root * root \leq n - a$

Decreases a

These changes forced the loop to fall under the appropriate constraints to keep my post conditions true. The code did not verify before because the variables could have breached these ranges, allowing the postcondition to be incorrect.

2g. After removing the precondition from the dafny code, I did not need to make any changes to the code in order for it to be verified. Removing the precondition allows negative whole numbers to be given as an input. However, due to my if statement annotation, this is accommodated for just the same as the standard input, allowing the code to verify as normal and still provide correct output.

2h.

Loop invariant: $(0 \leq a \leq n \vee 0 > a \geq n) \wedge \text{root} * \text{root} \leq n - a$

BASE CASE: does the LI hold before the loop?

$(0 \leq a \leq n \vee 0 > a \geq n) \wedge \text{root} * \text{root} \leq n - a$

The LI holds, given that $\text{root} == 0$ and $a == n$, $(n \geq 0) = (a \geq 0)$ and $(\text{root} \geq 0)$ by precondition

INDUCTION: assume the LI holds at iteration k, does it hold at iteration k+1?

- $a_new = a - (2 * \text{root} - 1)$
- $\text{root_new} = \text{root} + 1$
- $a_new = a - (2 * (\text{root} - 1) - 1) = a - (2 * \text{root} - 2 - 1) = a - (2 * \text{root} - 3)$
- root_new also holds, $0 \leq a \leq n \vee 0 > a \geq n$ and $\text{root} * \text{root} \leq n - a$ at iteration k.
- If $a \leq 0$ at iteration k, there would be no iteration k+1

$LI \wedge !b \Rightarrow \text{postcondition}$

- $(0 \leq a \leq n \vee 0 > a \geq n) \wedge \text{root} * \text{root} \leq n - a \wedge !(a > 0)$
- $\text{Root} * \text{root} \leq n - a \wedge a \leq 0 \ (0 \leq a \leq n \vee 0 > a \geq n)$
- $\text{Root} * \text{root} \leq n - a \wedge a \leq 0 \wedge 0 > a \geq n$
- $\Rightarrow \text{Root} * \text{root} \leq n - a \wedge 0 > a \geq n$ //by precondition

Does the loop terminate?

- Define $D == a - (2 * \text{root} - 1)$ // initially $a == n$, $\text{root} == 0$ and $a > 0$, $D > 0$
- D decreases with each step by 1 with each step
- D eventually reaches 0
- $D \leq 0 \Rightarrow$ loop exit condition $a == 0$
- When $D == 0$, loop exits

3a. Loop invariant: $\text{diffs.Length} == \text{arr.Length} - 1 \wedge a \leq \text{diffs.Length} \wedge \forall , \text{s.t. } 0 \leq k < a, \text{diffs}[k] = \text{arr}[k + 1] - \text{arr}[k]$

3b. BASE CASE: does the LI hold before the loop?

$\text{diffs.Length} == \text{arr.Length} - 1 \wedge a \leq \text{diffs.Length} \wedge \forall , \text{s.t. } 0 \leq k < a, \text{diffs}[k] = \text{arr}[k + 1] - \text{arr}[k]$

the LI holds, given that $a = 0$ and that no differences from any 2 values from the array arr have been added to diffs yet. diffs currently has no values in it, but is set at the size of 1 less than that of arr. $(a \leq \text{diffs.Length}) = (0 \leq \text{diffs.Length})$ by precondition

3c. INDUCTION: assume the LI holds at iteration k, does it hold at iteration k+1?

- $\text{diffs_new} == \text{new int}[\text{diffs.Length}]$
- $\text{diffs_new} == [\text{diffs}, \text{arr}[a + 1] - \text{arr}[a]] == [\text{arr}[1] + \text{arr}[0], \dots, \text{arr}[a + 1] - \text{arr}[a]]$
- $a_new == a + 1$
- $\text{diffs_new} == [\text{diffs}, \text{arr}[a_new] - \text{arr}[a_new - 1]] == [\text{arr}[1] + \text{arr}[0], \dots, \text{arr}[a_new] - \text{arr}[a_new - 1]]$
- $a_new \leq \text{diffs.Length}$ also holds; $a < \text{diffs.Length}$ at iteration k.
- $\text{Diffs_new.Length} == \text{diffs.Length}$ at iteration k.
- If $a == \text{diffs.Length}$ at iteration k, there would be no iteration k+1

3d. $LI \wedge !b \Rightarrow \text{postcondition}$

- $\text{diffs.Length} = \text{arr.Length} - 1 \wedge a \leq \text{diffs.Length} \wedge \forall, \text{ s.t. } 0 \leq k < a, \text{diffs}[k] = \text{arr}[k + 1] - \text{arr}[k] \wedge !(a < \text{diffs.Length})$
- $\Rightarrow \text{diffs.Length} = \text{arr.Length} - 1 \wedge (a == \text{diffs.Length}) \wedge \forall, \text{ s.t. } 0 \leq k < a, \text{diffs}[k] = \text{arr}[k + 1] - \text{arr}[k]$
- $\Rightarrow \text{diffs.Length} = \text{arr.Length} - 1 \wedge \forall, \text{ s.t. } 0 \leq k < \text{diffs.Length}, \text{diffs}[k] = \text{arr}[k + 1] - \text{arr}[k]$ //by precondition

3e. Does the loop terminate?

- Define $D == \text{diffs.Length} - a$ // initially $a == 0$ and $\text{diffs.Length} \geq 0$, $D \geq 0$
- Loop can be rewritten as: $(\text{While } D \geq 0) \{ D_new = \text{diff.Length} - (a + 1) = (\text{diff.Length} - a) - 1 = D - 1 \}$
- D decreases by 1 with each step.
- D eventually reaches 0.
- $D == 0 \Rightarrow$ loop exit condition $a == \text{diff.Length}$
- When $D == 0$, loop exits

4a.

Precondition: $\text{arr} != \text{null} \wedge \text{arr.Length} > 0 \wedge \forall j, \text{ s.t. } 0 \leq j < \text{arr.Length}, \text{arr}[j] == 'b' \vee \text{arr}[j] == 'r'$

$\text{Int } k = 0;$

$\text{Int } i = 0;$

```
while (i < arr.Length) {
    If arr[i] == 'r' {
        arr[i] = 'b';
        arr[k] = 'r';
        k = k + 1;
    }
    i = i + 1;
}
```

Postcondition: $0 \leq k \leq \text{arr.Length} \wedge \forall l, \text{ s.t. } 0 \leq l < k, \text{arr}[l] == 'r' \wedge \forall m, \text{ s.t. } k \leq m < \text{arr.Length}, \text{arr}[m] == 'b'$

4b. Postcondition: $0 \leq k \leq \text{arr.Length} \wedge \forall l, \text{s.t. } 0 \leq l < k, \text{arr}[l] == 'r' \wedge$
 $\forall m, \text{s.t. } k \leq m < \text{arr.Length}, \text{arr}[m] == 'b'$

4c.

Invariant: $i \leq \text{arr.Length} \wedge \forall n, \text{s.t. } 0 \leq n < \text{arr.Length}, \text{arr}[n] == 'r' \vee \text{arr}[n] == 'b'$
 $\wedge \forall o, \text{s.t. } k \leq o < i, \text{arr}[o] == 'b' \wedge \forall p, \text{s.t. } 0 \leq p < k, \text{arr}[p] == 'r'$